

0.0.1 Stochastic Process

A stochastic or random process, denoted by $\{s_t\}_{t \in T}$, can be defined as a collection of random variables that is indexed by some mathematical set T

- Index set T has the interpretation of time.
- The set T is typically $\mathbb{N}$ or $\mathbb{R}$.

0.1 Markov chain

0.1.1 State transition probability

For a markov state s and a successor state s' , the state transition probability is defined as

$$\mathcal{P}_{ss'} = \Pr\{s_{t+1} = s' | s_t = s\}$$

We define the state transition probability matrix as

$$\mathcal{P} = [\mathcal{P}_{ss'}]$$

0.2 Markov Reward Process (MRP)

A Markov Reward Process (MRP) is a Markov chain with an additional reward function. It is defined by the tuple $\langle \mathcal{S}, \mathcal{P}, \mathcal{R}, \gamma \rangle$, where:

- $\mathcal{S}$ is the state space.
- $\mathcal{P}$ is the state transition probability matrix.
- $\mathcal{R}$ is the reward function, with $\mathcal{R}(s_t) = r_{t+1}$ being the reward for being in state s_t at time t .
- $\gamma \in [0, 1)$ is the discount factor, which determines the importance of future rewards.

0.3 Policy evaluation

0.3.1 Value function with respect to a policy

We define $V^\pi(s)$ as the expected return (cumulative future reward) when starting from state s and following policy π :

$$V^\pi(s) = \mathbb{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^k r_{t+k+1} | s_t = s \right]$$

0.3.2 Decomposition of state value function

The state value function can be decomposed into the immediate reward and the expected value of the next state,

$$V^\pi(s) = \mathbb{E}_\pi [r_{t+1} + \gamma V^\pi(s_{t+1}) | s_t = s]$$

Expanding the expectation, with $\mathcal{R}_{ss'}^a = \mathcal{R}(s, a, s')$, we have

$$\begin{aligned}\mathbb{E}_\pi [r_{t+1}|s_t = s] &= \sum_a \pi(a|s) \sum_{s'} \mathcal{P}_{ss'}^a \mathcal{R}_{ss'}^a \\ \mathbb{E}_\pi [\gamma V^\pi(s_{t+1})|s_t = s] &= \sum_a \pi(a|s) \sum_{s'} \mathcal{P}_{ss'}^a V^\pi(s') \\ \implies V^\pi(s) &= \sum_a \pi(a|s) \sum_{s'} \mathcal{P}_{ss'}^a (\mathcal{R}_{ss'}^a + \gamma V^\pi(s'))\end{aligned}$$

The above is called the **Bellman Evaluation operator**.

0.3.3 Matrix formulation of Bellman Evaluation equation

We define,

$$\begin{aligned}\mathcal{P}^\pi(s'|s) &:= \sum_{a \in \mathcal{A}} \pi(a|s) \mathcal{P}_{ss'}^a \\ \mathcal{R}^\pi(s) &:= \sum_{a \in \mathcal{A}} \pi(a|s) \sum_{s'} \mathcal{P}_{ss'}^a \mathcal{R}_{ss'}^a = \mathbb{E}(r_{t+1}|s_t = s)\end{aligned}$$

Using the above definitions, for finite state MDP, we can rewrite the Bellman Evaluation equation in matrix form as

$$V^\pi = \mathcal{R}^\pi + \gamma \mathcal{P}^\pi V^\pi$$

or in explicit form,

$$V^\pi = (I - \gamma \mathcal{P}^\pi)^{-1} \mathcal{R}^\pi$$

0.4 Relation between Markov Decision Process and Markov Reward Process

MDP + Policy = MRP

0.5 Optimal policy

We define a partial ordering over policies,

$$\pi \geq \pi', \iff V^\pi(s) \geq V^{\pi'}(s) \quad \forall s \in \mathcal{S}.$$

0.5.1 Theorem: Existence of optimal policy

For any MDP, there exists an optimal policy π_* such that

$$V^{\pi_*}(s) \geq V^\pi(s) \quad \forall \pi \in \Pi, \forall s \in \mathcal{S}.$$

and all optimal policies achieve the same value function, i.e.,

$$V_*(s) = V^{\pi'_*}(s) \quad \forall \pi'_* \in \Pi_*.$$

0.6 Action value function

We define the action value function $Q^\pi(s, a)$ as the expected return when starting from state s , taking action a , and then following policy π :

$$Q^\pi(s, a) = \mathbb{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^k r_{t+k+1} \mid s_t = s, a_t = a \right]$$

We can decompose the action value function as follows

$$Q^\pi(s, a) = \mathbb{E}_\pi [r_{t+1} + \gamma V^\pi(s_{t+1}) \mid s_t = s, a_t = a]$$

Expanding the expectation, we have

$$Q^\pi(s, a) = \sum_{s'} \mathcal{P}_{ss'}^a \left(\mathcal{R}_{ss'}^a + \gamma \sum_{a'} \pi(a' \mid s') Q^\pi(s', a') \right)$$

0.7 Relationship between state ($V^\pi(\cdot)$) and action ($Q^\pi(\cdot, \cdot)$) value functions

The relationship between the state value function and the action value function is given by

$$V^\pi(s) = \sum_{a \in \mathcal{A}} \pi(a \mid s) Q^\pi(s, a)$$

and

$$Q^\pi(s, a) = \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^a (\mathcal{R}_{ss'}^a + \gamma V^\pi(s'))$$

As seen previously, all optimal policies achieve the optimal action value function,

$$Q_*(s, a) = \max_{\pi \in \Pi} Q^\pi(s, a)$$

and

$$Q_*(s, a) = Q^{\pi_*}(s, a) \quad \forall \pi_* \in \Pi_*$$

0.8 Greedy policy

For a given $Q^\pi(\cdot, \cdot)$, define $\pi'(s)$ as,

$$\pi'(s) = \text{greedy}(Q) = \begin{cases} 1 & \text{if } a = \arg \max_{a' \in \mathcal{A}} Q^\pi(s, a') \\ 0 & \text{otherwise} \end{cases}$$

For a given $V^\pi(\cdot)$, define $\pi'(s)$ as,

$$\pi'(s) = \text{greedy}(V) = \begin{cases} 1 & \text{if } a = \arg \max_{a \in \mathcal{A}} [\sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^a (\mathcal{R}_{ss'}^a + \gamma V^\pi(s'))] \\ 0 & \text{otherwise} \end{cases}$$

0.9 Relationship between optimal state ($V_*(\cdot)$) and optimal action ($Q_*(\cdot, \cdot)$) value functions

We have

$$V_*(s) = \max_{a \in \mathcal{A}} Q_*(s, a)$$

$$Q_*(s, a) = \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^a (\mathcal{R}_{ss'}^a + \gamma V_*(s'))$$

0.10 Policy Iteration

We have

$$\pi_0 \xrightarrow{E} v_{\pi_0} \xrightarrow{I} \pi_1 \xrightarrow{E} \dots \xrightarrow{I} \pi_* \xrightarrow{E} v_*,$$

where E is **Policy Evaluation** and I is **Policy Improvement**.

The full algorithm is as follows

Algorithm: Policy Iteration

Input: An MDP $\langle \mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma \rangle$ and an initial policy π_0

Output: An optimal policy π^*

1. Initialization

Set $k \leftarrow 0$

2. Repeat

a. (*Policy Evaluation*)

Compute the value function of the current policy π_k by solving the Bellman expectation equation for all $s \in \mathcal{S}$:

$$V^{\pi_k}(s) \leftarrow \sum_{a \in \mathcal{A}} \pi_k(a|s) \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^a [R_{ss'}^a + \gamma V^{\pi_k}(s')]$$

b. (*Policy Improvement*)

Improve the policy by acting greedily with respect to V^{π_k} for all $s \in \mathcal{S}$:

$$\pi_{k+1}(s) \leftarrow \arg \max_{a \in \mathcal{A}} \left(\sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^a [R_{ss'}^a + \gamma V^{\pi_k}(s')] \right)$$

c. Set $k \leftarrow k + 1$

Until $\pi_k(s) = \pi_{k-1}(s)$ for all $s \in \mathcal{S}$ (policy is stable)

3. Return $\pi^* \leftarrow \pi_k$

0.11 Value Iteration

Algorithm: Value Iteration

Input: An MDP $\langle \mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma \rangle$ and an value function V_0

Output: An optimal policy π^*

1. Initialization

Set $k \leftarrow 0$

2. Repeat

a. for $s \in \mathcal{S}$

$$V_{k+1}(s) \leftarrow \max_{a \in \mathcal{A}} \left[\sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^a (\mathcal{R}_{ss'}^a + \gamma V_k(s')) \right]$$

b. Set $k \leftarrow k + 1$

Until $V_k(s) = V_{k-1}(s)$ for all $s \in \mathcal{S}$ (value function is stable)

3. **Return** $\pi^* \leftarrow \arg \max_{a \in \mathcal{A}} \left[\sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^a (\mathcal{R}_{ss'}^a + \gamma V_k(s')) \right]$

0.11.1 Value function space

Let

$$\mathcal{V} = \{f : \mathcal{S} \rightarrow \mathbb{R} \mid \sup_{s \in \mathcal{S}} |f(s)| < \infty\},$$

be the space of all value functions.

0.11.2 Bellman Evaluation Operator

We define Bellman Evaluation Operator $\mathcal{L}^\pi : \mathcal{V} \rightarrow \mathcal{V}$ as,

$$\mathcal{L}^\pi(v) = \mathcal{R}^\pi + \gamma \mathcal{P}^\pi v$$

0.11.3 Bellman Optimality Operator

We define Bellman Optimality Operator $\mathcal{L} : \mathcal{V} \rightarrow \mathcal{V}$ as,

$$\mathcal{L}(v) = \max_{a \in \mathcal{A}} \left[\mathcal{R}^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^a v \right]$$

0.12 Prediction and Control in Reinforcement Learning

1 Pictures

Algorithm Value Iteration

- 1: Start with an initial value function $V_1(\cdot)$;
- 2: **for** $k = 1, 2, \dots, K$ **do**
- 3: **for** $s \in \mathcal{S}$ **do**
- 4: Calculate

$$V_{k+1}(s) \leftarrow \max_a \left[\sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^a (\mathcal{R}_{ss'}^a + \gamma V_k(s')) \right]$$

- 5: **end for**
 - 6: **end for**
-

Algorithm Policy Iteration

- 1: Start with an initial policy π_1
- 2: **for** $i = 1, 2, \dots, N$ **do**
- 3: Evaluate $V^{\pi_i}(s) \quad \forall s \in \mathcal{S}$. That is,
- 4: **for** $k = 1, 2, \dots, K$ **do**
- 5: For all $s \in \mathcal{S}$ calculate

$$V_{k+1}^{\pi_i}(s) \leftarrow \sum_a \pi(a|s) \sum_{s'} \mathcal{P}_{ss'}^a [\mathcal{R}_{ss'}^a + \gamma V_k^{\pi_i}(s')]$$

- 6: **end for**
- 7: Perform policy Improvement

$$\pi_{i+1} = \text{greedy}(V^{\pi_i})$$

- 8: **end for**
-

There is a recursive formulation for $Q_*(\cdot, \cdot)$

$$Q_*(s, a) = \left[\sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^a \left(\mathcal{R}_{ss'}^a + \gamma \max_{a'} Q_*(s', a') \right) \right]$$

One could similarly conceive an iterative algorithm to compute optimal Q_* using the above recursive formulation !!

► Prediction

- ★ Input MDP ($\langle \mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma \rangle$) and policy π
- ★ Output : $V^\pi(\cdot)$

► Control

- ★ Input MDP ($\langle \mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma \rangle$)
- ★ Output : Optimal value function $V_*(\cdot)$ or optimal policy π_*

First-visit Monte Carlo Policy Evaluation



- To evaluate $V^\pi(s)$ for some given state s , repeat over several episodes
 - ★ The **first** time t that $s_t = s$ in the episode
 1. Increment counter for number of visits to s : $N(s) \leftarrow N(s) + 1$
 2. Increment running sum of total returns with return from current episode:
 $S(s) \leftarrow S(s) + G_t$
 - Monte Carlo estimate of value function $V(s) \leftarrow S(s)/N(s)$

By the law of large numbers $V(s) \rightarrow V^\pi(s)$ as number of episodes increases

Monte Carlo Method : Example

- ▶ Consider an MDP with two states $\mathcal{S} = \{A, B\}$ with $\gamma = 1$
- ▶ $\mathcal{P}$ and $\mathcal{R}$ are unknown
- ▶ Consider a policy π that gives rise to following state-reward sequence
 - ★ $A(+3), A(+2), B(-4), A(+4), B(-3)$
 - ★ $B(-2), A(+3), B(-3)$
- ▶ What is $V^\pi(A)$ and $V^\pi(B)$ if we use first visit MC ?
- ▶ First visit MC : $V(A) = \frac{1}{2}(2 + 0) = 1$; $V(B) = \frac{1}{2}(-3 - 2) = -5/2$

Every-visit Monte Carlo Policy Evaluation

- ▶ To evaluate $V^\pi(s)$ for some given state s , repeat over several episodes
 - ★ **Every** time t that $s_t = s$ in the episode
 1. Increment counter for number of visits to s : $N(s) \leftarrow N(s) + 1$
 2. Increment running sum of total returns with return from current episode:
 $S(s) \leftarrow S(s) + G_t$
 - ▶ Monte Carlo estimate of value function $V(s) \leftarrow S(s)/N(s)$
- By the law of large numbers $V(s) \rightarrow V^\pi(s)$ as number of episodes increases

Monte Carlo Method : Example



- ▶ Consider an MDP with two states $\mathcal{S} = \{A, B\}$ with $\gamma = 1$
- ▶ $\mathcal{P}$ and $\mathcal{R}$ are unknown
- ▶ Consider a policy π that gives rise to following state-reward sequence
 - ★ $A(+3), A(+2), B(-4), A(+4), B(-3)$
 - ★ $B(-2), A(+3), B(-3)$
- ▶ What is $V^\pi(A)$ and $V^\pi(B)$ if we use first visit MC and every visit MC respectively ?
- ▶ First visit MC : $V(A) = \frac{1}{2}(2 + 0) = 1$; $V(B) = \frac{1}{2}(-3 - 2) = -5/2$
- ▶ Every visit MC : $V(A) = \frac{1}{4}(2 - 1 + 1 + 0) = 1/2$; $V(B) = \frac{1}{4}(-3 - 3 - 3 - 2) = -11/4$

One-Step TD



- ▶ We wish to approximate

$$V^\pi(s) = \mathbb{E}_\pi [r_{t+1} + \gamma V^\pi(s_{t+1}) | s_t = s]$$

- ▶ Approximate the expectation by a sample mean
 - ★ If the *transition* (s_t, r_{t+1}, s_{t+1}) is observed at time t under π , then

$$V(s_t) \leftarrow V(s_t) + \alpha_t [r_{t+1} + \gamma V(s_{t+1}) - V(s_t)]$$

- ★ Samples come from different visits to the state s , either from same or different trajectories
- ★ Compute the sample mean incrementally

One-Step TD : TD(0) Algorithm



Algorithm TD(0) : Algorithm

- 1: Initialize $V(s)$ arbitrarily (say, $V(s) = 0 \quad \forall s \in \mathcal{S}$);
- 2: **for** $k = 1, 2, \dots, K$ **do**
- 3: Let s be a start state for episode k
- 4: **for** For each step in the k -th episode **do**
- 5: Take action a recommended by policy π from state s
- 6: Collect reward r and reach next state s'
- 7: Perform the following TD update

$$V(s) = V(s) + \alpha[r + \gamma V(s') - V(s)]$$

- 8: Assign $s \leftarrow s'$
 - 9: **end for**
 - 10: **end for**
-

Convergence of TD Algorithms



- For any fixed policy π , the TD(0) algorithm described above converges (asymptotically) to V^π under some conditions on the choice of α (Robbins Monroe Condition)

- ★ $\sum \alpha_t = \infty$
- ★ $\sum \alpha_t^2 < \infty$

- *Generally*, TD methods have usually been found to converge faster than MC methods on certain class of tasks

Connections between MC Error and TD Error

- ▶ The term $\delta_t = [r_{t+1} + \gamma V(s_{t+1}) - V(s_t)]$ is called the (one step) **TD error**
- ▶ The term $G_t - V(s_t)$ is called the **MC error**
- ▶ If a trajectory has T time steps then

$$G_t - V(s_t) = \sum_{k=0}^{T-t-1} \gamma^k \delta_{t+k}$$

(Exercise : Prove it !)

Multi-step TD

- ▶ One-step TD

$$\begin{aligned} V^\pi(s) &= \mathbb{E}_\pi [r_{t+1} + \gamma V^\pi(s_{t+1}) | s_t = s] \\ V(s_t) &\leftarrow V(s_t) + \alpha_t [r_{t+1} + \gamma V(s_{t+1}) - V(s_t)] \end{aligned}$$

- ▶ Two-step TD

$$\begin{aligned} V^\pi(s) &= \mathbb{E}_\pi [r_{t+1} + \gamma r_{t+2} + \gamma^2 V^\pi(s_{t+2}) | s_t = s] \\ V(s_t) &\leftarrow V(s_t) + \alpha_t [r_{t+1} + \gamma r_{t+2} + \gamma^2 V(s_{t+2}) - V(s_t)] \end{aligned}$$

- ▶ More generally, define the n -step return

$$G_t^{(n)} \stackrel{\text{def}}{=} r_{t+1} + \gamma r_{t+2} + \dots + \gamma^{n-1} r_{t+n} + \gamma^n V(s_{t+n})$$

- ▶ n -step TD

$$V(s_t) \leftarrow V(s_t) + \alpha_t [G_t^{(n)} - V(s_t)]$$

λ-Return

- Any set of returns can be averaged, even an infinite set, as long as the weights on the component returns are positive and sum to 1
- Choose $\lambda \in [0, 1]$, and define the λ -return at time t as

$$G_t^\lambda \stackrel{\text{def}}{=} (1 - \lambda) \sum_{n=1}^{\infty} \lambda^{n-1} G_t^{(n)}$$

- λ -return algorithm: use λ -return as the target

$$V(s_t) \leftarrow V(s_t) + \alpha_t [G_t^\lambda - V(s_t)]$$

- If the episode ends at $T > t$, define $G_t^{(n)}$ to be G_t for all $n > T - t$. Then

$$G_t^\lambda = (1 - \lambda) \sum_{n=1}^{T-t} \lambda^{n-1} G_t^{(n)} + \lambda^{T-t} G_t$$

- $\lambda = 0$ gives 1-step TD, $\lambda = 1$ gives MC update

A Rearrangement of n -step TD Errors

$$\begin{aligned} \delta_t^{(n)} &\stackrel{\text{def}}{=} r_{t+1} + \gamma r_{t+2} + \cdots + \gamma^{n-1} r_{t+n} + \gamma^n V(s_{t+n}) - V(s_t) \\ &= \gamma^0 [r_{t+1} + \gamma V(s_{t+1}) - V(s_t)] \\ &\quad + \gamma^1 [r_{t+2} + \gamma V(s_{t+2}) - V(s_{t+1})] \\ &\quad \vdots \\ &\quad + \gamma^{n-1} [r_{t+n} + \gamma V(s_{t+n}) - V(s_{t+n-1})] \\ &= \sum_{i=t}^{t+n-1} \gamma^{i-t} \delta_i^{(1)} \end{aligned}$$

$$\begin{aligned} G_t^\lambda - V(s_t) &= (1 - \lambda) \sum_{n=1}^{\infty} \lambda^{n-1} \delta_t^{(n)} = (1 - \lambda) \sum_{n=1}^{\infty} \lambda^{n-1} \sum_{i=t}^{t+n-1} \gamma^{i-t} \delta_i^{(1)} \\ &= \sum_{i=0}^{\infty} (\lambda \gamma)^i \delta_{t+i}^{(1)} \end{aligned}$$

Algorithm : TD(λ)



Algorithm TD(λ) : Algorithm

```
1: Initialize  $e(s) = 0$  for all  $s$ ,  $V(s)$  arbitrarily
2: for For each episode do
3:   Let  $s$  be a start state for episode  $k$ 
4:   for For each step of the episode do
5:     Take action  $a$  recommended by policy  $\pi$  from state  $s$ 
6:     Collect reward  $r$  and reach next state  $s'$ 
7:     Form the one-step TD error  $\delta \leftarrow r + \gamma V(s') - V(s)$ 
8:     Increment eligibility trace of state  $s$ ,  $e(s) \leftarrow e(s) + 1$ 
9:     for For all states  $S \in \mathcal{S}$  do
10:      Update  $V(S)$ :  $V(S) \leftarrow V(S) + \alpha e(S) \delta$ 
11:      Update eligibility trace:  $e(S) \leftarrow \lambda \gamma e(S)$ 
12:    end for
13:    Move to next state:  $s \leftarrow s'$ 
14:  end for
15: end for
```

Certainty Equivalence Estimate



- ▶ Model based policy evaluation with model estimated from samples
- ▶ Given an experience quadruple (s, a, r, s') , we can estimate
 - ★ Compute MLE for model using (s, a, s')

$$\hat{P}(s'|s, a) = \frac{1}{N(s, a)} \sum_{k=1}^K \sum_{t=1}^{L_k-1} \mathbb{1}(s_{k,t} = s, a_{k,t} = a, s_{k,t+1} = s')$$

- ★ Compute MLE estimate of the reward function

$$\hat{R}(s, a, s') = \frac{1}{N(s, a, s')} \sum_{k=1}^K \sum_{t=1}^{L_k-1} \mathbb{1}(s_{k,t} = s, a_{k,t} = a, s_{k,t+1} = s') r_{t,k}$$

- ▶ Once MLE estimates of $\hat{P}(s'|s, a)$ and $\hat{R}(s, a, s')$ use a known policy evaluation method to compute a MLE based estimate for V^π .

- ▶ Can we estimate the value V^π or Q^π of a *target policy* π ...
- ▶ While following a *behavior policy* μ ?
 - ★ Trajectories or transitions are sampled from μ
 - ★ Expected values have to be estimated w.r.t. π
- ▶ Possible benefits:
 - ★ Learn by observing other agents
 - ★ Re-use previous experience generated from earlier policies
 - ★ Learn about optimal policy while following exploratory policy
 - ★ Learn about multiple policies while following one policy

Importance Sampling : Review

Let $P(x)$ be the target distribution and $Q(x)$ be the behaviour distribution for some random variable x

$$\begin{aligned}\mathbb{E}_{X \sim P}[f(X)] &= \sum_i P(x_i) f(x_i) \\ &= \sum_i Q(x_i) \left[\frac{P(x_i)}{Q(x_i)} f(x_i) \right] \\ &= \mathbb{E}_{X \sim Q} \left[\frac{P(X)}{Q(X)} f(X) \right]\end{aligned}$$

- ▶ We have samples of X drawn from Q , but we wish to estimate expectation under P

$$\mathbb{E}_{X \sim P}[f(X)] \simeq \frac{1}{n} \sum_{i=1}^n \left[\frac{P(X_i)}{Q(X_i)} f(X_i) \right], \quad X_i \sim Q$$

- ▶ Caveat: Q should not assign zero probability to any outcome that is assigned non-zero probability by P

Importance Sampling : Review



The ratio $\frac{P(x)}{Q(x)}$ is the importance sampling (IS) weight for x .

What about the variance of IS estimator $\hat{\mu}_Q \approx \frac{1}{N} \sum_{x_i \in D} \left[\frac{P(x_i)}{Q(x_i)} f(x_i) \right]$?

$$\begin{aligned} \text{var}_Q [\hat{\mu}_Q] &= \text{var}_Q \left[\frac{P(x)}{Q(x)} f(x) \right] \\ &= \left[\mathbb{E}_Q \left[\left(\frac{P(x)}{Q(x)} f(x) \right)^2 \right] - \mathbb{E}_Q \left(\left[\frac{P(x)}{Q(x)} f(x) \right] \right)^2 \right] \\ &= \mathbb{E}_P \left[\left(\frac{P(x)}{Q(x)} f(x) \right)^2 \right] - \mathbb{E}_P (f(x))^2 \end{aligned}$$

If the likelihood ratio $\frac{P(x)}{Q(x)}$ is large, the variance of the estimator explodes

Off-Policy TD using Importance Sampling



- ▶ Evaluate target policy π using TD targets $r_{t+1} + \gamma V(s_{t+1})$ generated from μ
- ▶ Weigh each TD target by the importance sampling factor $\pi(s_t, a_t) / \mu(s_t, a_t)$

$$V(s_t) \leftarrow V(s_t) + \alpha_t \left[\frac{\pi(s_t|a_t)}{\mu(s_t|a_t)} (r_{t+1} + \gamma V(s_{t+1})) - V(s_t) \right]$$

- ▶ π may be deterministic and greedy, μ should be stochastic and exploratory
- ▶ The case $\mu = \pi$ is called *on-policy* learning
- ▶ **On Policy Learning** : Learn about policy π from experience sampled from π
- ▶ **Off Policy Learning** : Learn about policy π from experience sampled from μ

Algorithm Monte Carlo Control : GLIE

- 1: Initialize $Q(s,a) = 0$, set $\varepsilon = 1$;
- 2: Create an ε -greedy initial policy π_1 and choose a schedule for ε -decay (eg. linear decay)
- 3: **for** $k = 1, 2, \dots, K$ **do**
- 4: Sample a trajectory from policy π_k
- 5: **for** For each state action (s_t, a_t) pair in the trajectory **do**
- 6: Compute the total discounted return G_t starting from (s_t, a_t)
- 7:
$$N(s_t, a_t) = N(s_t, a_t) + 1$$
- 8:
$$Q(s_t, a_t) = Q(s_t, a_t) + \frac{1}{N(s_t, a_t)} (G_t - Q(s_t, a_t))$$
- 9: **end for**
- 10: Perform the policy improvement step as $\pi_{k+1} = \epsilon$ -greedy(Q^{π_k})
- 11: Decay ε according to the decaying schedule
- 12: **end for**

TD Evaluation of Q Function

- State-action value function of a policy π :

$$\begin{aligned} Q^\pi(s, a) &\stackrel{\text{def}}{=} \mathbb{E}_\pi(G_t | s_t = s, a_t = a) \\ &= \mathbb{E}_\pi(r_{t+1} + \gamma Q^\pi(s_{t+1}, a_{t+1}) | s_t = s, a_t = a) \end{aligned}$$

- Iterative DP policy evaluation:

$$\begin{aligned} Q_{k+1}(s, a) &\leftarrow \sum_{s'} \mathcal{P}_{ss'}^a \left[\mathcal{R}_{ss'}^a + \gamma \sum_{a'} (\pi(s', a') Q_k(s', a')) \right] \\ Q_k &\rightarrow Q^\pi \end{aligned}$$

- TD approximation: Given the transition $(s_t, a_t, r_{t+1}, s_{t+1})$, sample $a' \sim \pi(s_{t+1}, \cdot)$, and update

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha_t [r_{t+1} + \gamma Q(s_{t+1}, a') - Q(s_t, a_t)]$$

TD Evaluation of Q Function : SARSA



- ▶ TD approximation: Given the transition $(s_t, a_t, r_{t+1}, s_{t+1})$, sample $a' \sim \pi(s_{t+1}, \cdot)$, and perform the following update

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha_t [r_{t+1} + \gamma Q(s_{t+1}, a') - Q(s_t, a_t)]$$

- ▶ On-policy version (SARSA): $a_t \sim \pi(s_t, \cdot)$
- ▶ Off-policy version: $a_t \sim \mu(s_t, \cdot)$;
 - ★ Need to multiply the term inside square brackets with suitable importance sampling factor

TD Evaluation : Convergence



- ▶ On Policy and off-policy version converges to Q^π
 - ★ Convergence takes place under similar conditions as TD methods for V^π
 - ▶ State and action spaces are finite
 - ▶ All state-action pairs are visited infinitely often
 - ▶ Robbins-Monroe condition: $\sum_t \alpha_t = \infty, \sum_t \alpha_t^2 < \infty$

SARSA: On-Policy Control



- Policy is always ε -greedy with ε decay
- Given a trajectory segment $(s_t, a_t, r_{t+1}, s_{t+1}, a_{t+1})$ generated by the ε -greedy policy, update

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha_t[r_{t+1} + \gamma Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t)]$$

Algorithm SARSA

- 1: Initialize $Q(s, a)$ arbitrarily, with Q at terminal states set to zero
 - 2: **for** Repeat for each episode **do**
 - 3: Initialize s , choose action a at s using ε -greedy over Q
 - 4: **for** Repeat for each step in the episode **do**
 - 5: Take action a , observe reward r and next state s'
 - 6: Choose action a' for state s' using ε -greedy over Q
 - 7: $Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma Q(s', a') - Q(s, a)], s \leftarrow s', a \leftarrow a'$
 - 8: **end for**
 - 9: **end for**
-

Learning Optimal State-Action Value Function



- Optimal Q function:

$$Q_*(s, a) \stackrel{\text{def}}{=} \max_{\pi} Q^{\pi}(s, a) = Q^{\pi^*}(s, a)$$

- Bellman optimality equation:

$$Q_*(s, a) = \mathbb{E} \left[r_{t+1} + \gamma \max_{a'} Q_*(s_{t+1}, a') \mid s_t = s, a_t = a \right]$$

- Iterative DP approximation

$$Q_{k+1}(s, a) \leftarrow \sum_{s'} \mathcal{P}_{ss'}^a \left[\mathcal{R}_{ss'}^a + \gamma \max_{a'} Q_k(s', a') \right]$$

$$Q_k \rightarrow Q_*$$

Q-Learning: Off-Policy Control



- Policy is always ε -greedy with ε decay
- Given a trajectory segment $(s_t, a_t, r_{t+1}, s_{t+1})$ generated by the ε -greedy policy, update

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha_t [r_{t+1} + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t)]$$

Algorithm Q-Learning

- 1: Initialize $Q(s, a)$ arbitrarily, with Q at terminal states set to zero
 - 2: **for** Repeat for each episode **do**
 - 3: Initialize s , choose action a at s using ε -greedy over Q
 - 4: **for** Repeat for each step in the episode **do**
 - 5: Take action a , observe reward r and next state s'
 - 6: Choose target to update $Q(s, a)$ by being greedy at s' as shown below
 - 7:
$$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma \max_{a'} Q(s', a') - Q(s, a)], s \leftarrow s'$$
 - 8: **end for**
 - 9: **end for**
-

Easawr Subramanian, IIT Hyderabad

30 of 39



2 Function approximation methods

2.1 Monte Carlo based value function fitting

Use Monte Carlo to get rollouts
Perform supervised regression

$$y_i = \sum_{k=0}^{\infty} (\gamma^k r_{t+k+1} \mid s_t = s)$$
$$(s_i, y_i)$$
$$L(\phi) = \frac{1}{2} \sum_N^{i=1} (V_{\phi}^{\pi}(s_i) - y_i)^2$$

2.1.1 See also

2.1.2 References